Empirical Paradigms in Browser Extension Security: Dataset Inventories, Versioning Acquisition, and Delta Analysis Methodologies

The modern web browser has evolved from a simple document viewer into a highly complex, pseudo-operating system, capable of executing sophisticated applications directly on the client side. At the core of this extensibility is the browser extension ecosystem—a framework of HTML, CSS, JavaScript, and WebAssembly components that interact with high-privilege browser APIs. While these add-ons enhance user productivity, their privileged position makes them a prime vector for malicious actors. Extensions operate across multiple execution contexts, including content scripts injected into web pages, background service workers managing state and API calls, and isolated extension pages[1]. Through these mechanisms, extensions can bypass the Same-Origin Policy (SOP), intercept unencrypted network traffic, modify the Document Object Model (DOM), and exfiltrate sensitive session tokens before Transport Layer Security (TLS) encryption is applied[3].

The transition to Manifest V3 (MV3) in Google Chrome, alongside parallel architectural shifts in Mozilla Firefox, has significantly altered the threat landscape by deprecating powerful APIs in favor of declarative and restrictive alternatives[4]. However, empirical research into browser extension malware relies heavily on access to robust, accurately labeled datasets and the ability to track extension codebases across consecutive updates. Because extensions update automatically and silently in the background, a benign extension can easily be transformed into malware post-installation via a malicious update, a phenomenon often resulting from a developer selling a legitimate extension to a malicious actor or a supply-chain compromise[3].

To investigate these phenomena, the security community requires two distinct types of empirical resources: massive, labeled corpora of known malicious and benign extensions for training classifiers, and reliable acquisition pipelines to download historical and consecutive versions of specific extension identifiers for delta analysis. This report provides an exhaustive inventory of publicly available extension datasets, details reliable methodologies for acquiring consecutive extension versions to facilitate code delta comparisons, and synthesizes the underlying trends driving contemporary extension security research.

Compendium of Publicly Available Extension Datasets

The availability of high-quality, labeled datasets has historically been a bottleneck in browser extension security research. While the Chrome Web Store hosts over 200,000 extensions[7], public corpora of confirmed malicious samples remain sparse. This scarcity is primarily driven by the rapid takedown of malicious extensions by browser vendors, which limits the window of opportunity for researchers to acquire the raw extension packages (CRX or XPI files), as well as the legal and ethical complexities associated with redistributing active malware. Consequently,

researchers must rely on a patchwork of community-maintained Indicator of Compromise (IOC) databases, academically curated historical archives, and specialized threat-specific datasets. The following inventory details every publicly available dataset, categorized by their structural focus, size, labeling taxonomy, licensing, and acquisition vectors.

The Malicious Chrome Extension IOC Database (chrome-mal-ids)

Maintained by The Privacy Commons Institute (TPCI), this repository represents the most active, community-driven database of malicious Chrome and Edge extension Indicators of Compromise (IOCs) currently available[6]. Originating in 2021 as an independent research project designed to fill the void of authoritative extension ID lists, the database now tracks thousands of documented malicious extensions across dozens of campaigns. These campaigns range from credential stealers and browser hijackers to sophisticated ad fraud rings and supply chain compromises[6]. The dataset does not state a fixed numeric count in its primary documentation because the total continuous ingestion changes numbers frequently; instead, live IOC counts and campaign breakdowns are auto-generated on every commit inside the repository's statistical trackers[6].

The dataset is highly structured, having undergone a major schema update in May 2026 to incorporate the TPCI-V multi-stage verification protocol, which adds significant granularity to the labeling process[6]. This protocol ensures a rigorous classification of threats, dividing entries into two primary confidence levels. The first level, "Independently Verified Entries," comprises data sourced from published security research that has been individually human-reviewed prior to publication, complete with source citations and campaign attribution[6]. The second level, "Third-Party Source Entries," includes data ingested via bulk imports (such as Delta_Import or csv_import) and feeds, which are subjected to a Stage 5A static behavioral analysis rather than individual initial human review[6].

The underlying schema of the database includes critical metadata such as the 32-character extension ID (EXTID), the extension display name, the date the malicious behavior was first reported (DATE-DIS), the type of threat (THREAT-TYPE), and the targeted browser[6]. Crucially for longitudinal research, the dataset includes an OWNERSHIP-TRANSFER flag to denote when a legitimate extension was acquired and subsequently weaponized, as well as a STILL-ACTIVE flag to indicate whether the extension was live in the browser store at the time of reporting[6]. Furthermore, the dataset tracks supply chain victims through the TPCI-IDENTITY status label, marking extensions as "remediated" if developers have successfully patched the malicious code injected by third-party compromises; these IDs are retained for historical accuracy but are not treated as currently malicious[6]. The dataset is distributed under a Creative Commons Attribution 4.0 International (CC BY 4.0) license and is available in standard CSV formats as well as an automated MISP (Malware Information Sharing Platform) feed for seamless integration into enterprise Security Information and Event Management (SIEM) systems[6].

The "You've Changed" Extension Deltas Corpus

Developed for the seminal 2020 ACM SIGSAC (CCS) paper *You've Changed: Detecting Malicious Browser Extensions through their Update Deltas*, this dataset is the largest publicly available academic corpus designed specifically for version-to-version delta analysis[8]. The core

premise of the research was that static analysis of a single extension version is insufficient for detecting malware that employs delayed activation or timebombs; instead, analyzing the abstract syntax trees, API sequences, and permission changes between consecutive versions provides a much higher fidelity signal of malicious intent[9].

To build this corpus, the researchers established a massive crawling infrastructure, analyzing an initial pool of 152,000 extensions and ultimately collecting a staggering 922,684 individual extension versions to track code changes over time[8]. The public repository associated with the research contains the underlying data utilized for their novel anomaly detection mechanisms. Specifically, the researchers utilized Natural Language Processing (NLP) on user reviews in the Chrome Web Store, identifying negative rating spikes as a primary indicator that a previously benign extension had turned malicious following an update[8]. The released dataset explicitly includes 5,151 Chrome extensions represented by forty static features, alongside the historical API sequence delta data used to train the machine learning classifiers[9].

The labeling in this dataset is highly contextual, categorizing extensions as benign or malicious based on rating anomalies, clustering, and the extraction of API sequence deltas that deviate from established baselines[9]. Notably, the researchers found that 83% of the infringing extensions identified were New Tab Extensions (NTEs), highlighting a specific vector heavily abused by threat actors[11]. The dataset and the accompanying analysis code (written primarily in Python and R) are open-source and freely available to academic researchers via the associated GitHub repository[9].

WasmMal-15K: WebAssembly Malware Dataset

As browser extensions increasingly incorporate WebAssembly (Wasm)—a binary instruction format designed for efficient cross-platform execution—malicious actors have adapted their strategies to utilize Wasm for obfuscating malicious logic. Traditional JavaScript static analysis engines frequently fail to parse or evaluate raw binary Wasm components, allowing cryptojackers and stealthy data exfiltration routines to operate undetected[13].

To address this emerging threat vector, the *WasmMal-15K* dataset was curated and released in conjunction with the WasmGuard detection framework, presented at the ACM Web Conference 2025 (WWW '25). The dataset is massive in scale for this specific niche, containing a total of 15,024 Wasm binaries perfectly balanced between 7,512 benign and 7,512 malicious samples[13]. The labeling is a strict binary classification based on the presence of malicious binary execution patterns.

The accompanying research highlights the necessity of this dataset by demonstrating that previous deep learning-based detection methods suffered critical performance degradation under adversarial conditions, dropping to 49.90% accuracy[13]. By utilizing the WasmMal-15K dataset, the researchers employed Fast Gradient Sign Method (FGSM)-based adversarial training to achieve up to 99.20% robust accuracy against Projected Gradient Descent (PGD-50) adversarial attacks[13]. The dataset is open-source and hosted on GitHub, providing a critical resource for researchers analyzing how extension malware utilizes raw-binary execution to evade standard browser security controls[13].

Generative AI Malicious Extensions Corpus

The rapid proliferation of Generative AI has spawned an entirely new category of browser extensions offering text summarization, intelligent translation, and context-aware writing assistance directly within the user's workflow[2]. Threat actors have rapidly capitalized on this trend, creating malicious extensions that masquerade as AI assistants to intercept the highly sensitive contextual data (emails, internal documents, proprietary code) that users willingly feed into these tools[15].

A comprehensive 2025 academic dataset provides ground-truth data on 341 confirmed malicious GenAI-themed Chrome extensions[15]. The compilation consists of two distinct segments: 187 extensions confirmed through public disclosures and existing academic literature, and 154 novel malicious extensions detected through a targeted internal scan of 5,551 AI-themed extensions released between January and September 2025[15]. The labeling taxonomy differentiates between extensions flagged via public disclosure and those uncovered through novel detection methodologies[15].

The vulnerabilities inherent to these AI extensions are fundamentally different from traditional DOM-manipulation malware. Research utilizing the *Assure* testing framework on AI extensions found an average of 88.5 distinct security issues per extension[17]. A primary vulnerability is prompt injection; if an AI summarizer extension reads a web page containing hidden, malicious prompt instructions (e.g., invisible text dictating the Large Language Model to exfiltrate session tokens to an external server), the extension will blindly process the attack[17]. The datasets tracking these GenAI extensions highlight a paradigm shift, proving that extensions powered by LLMs break the traditional boundaries of predictable execution and require robust input sanitization and "visible-only processing" filters[17].

Auxiliary Integration Datasets (SmartShield and Malicious URLs)

While not exclusively datasets of extension files, several prominent machine learning datasets are inextricably linked to browser extension malware research, as they provide the underlying training data for extensions designed to detect malicious activity in real-time. For instance, the *SmartShield* repository utilizes the Malimg dataset (a dataset of malware images) to train a ResNet50 and Support Vector Machine (SVM) ensemble model[18]. This model is integrated directly into a browser extension that performs client-side, real-time scanning of images to detect steganographically embedded threats[18].

Similarly, extension-based malicious URL detectors frequently rely on massive external corpora, such as a 651,191-record dataset aggregated from ISCX URL 2016, PhishTank, and the Malware Domain List[19]. These datasets provide the 27 advanced URL characteristics (length, hostnames, special character counts) required to train the Random Forest classifiers operating within the extension's background scripts, allowing for real-time threat classification into benign, phishing, malware, or defacement categories[19].

Historical Risk Assessment Repositories (CRXcavator and DeDup.js)

For several years, CRXcavator, operated by Duo Labs, served as the premier API and dataset for extension risk scoring. It provided automated risk assessments for over 550,000 extensions across Chrome, Firefox, and Edge, evaluating permissions, external domain communications, and the presence of vulnerable third-party libraries[20]. CRXcavator was a critical resource for

enterprise administrators, offering numerical risk scores and identifying exploitable vulnerabilities[20].

However, CRXcavator ceased operations in late 2024, leaving a significant void in the research community[21]. While commercial alternatives like Spin.AI have absorbed much of the enterprise market, open-source replacements such as the crx_checker Python script have emerged[20]. This script utilizes VirusTotal APIs and the Retire.js library for Software Bill of Materials (SBOM) scanning, attempting to replicate CRXcavator's functionality locally by computing SHA-256 hashes and extracting CRX contents for risk assessment[21].

Another vital historical analysis tool is the DeDup.js framework, which utilized similarity analysis to detect potentially malicious extensions during the approval process[22]. By generating hashes for the static resources (HTML, CSS, JS) of browser extensions and maintaining a tracking database, DeDup.js uncovered that 86% of newly infringing extensions were structurally identical to items previously banned by Google[11]. The framework was instrumental in identifying the CacheFlow attack, successfully detecting 53 malicious extensions that were duplicating malicious behaviors while attempting to evade static vetting[22].

Structured Inventory of Public Extension Datasets

Dataset Name / Repository	Size / Volume	Labeling / Schema	License / Access	Primary URL / Source
chrome-mal-ids (TPCI)	Thousands of live IOCs (continuous updates via stats tracking)	THREAT-TYPE, STILL-ACTIVE, TPCI-V Verification (5 stages), OWNERSHIP-TRANSFER	CC BY 4.0	github.com/mallorybowes/chrome-mal-ids [cite: 6]
Extension Deltas ('You've Changed')	152k base extensions, 922k historical versions, 5,151 static feature records	Benign vs. Malicious based on rating anomalies, API sequence deltas, and clustering	Open Source (Academic)	github.com/wspr-ncsu/extensiondeltas [cite: 9]
WasmMal-15K	15,024 Wasm binaries (7,512 benign, 7,512	Binary classification (Malicious /	Open Source (Academic)	github.com/Yuxia-Sun/WasmMal

	malicious)	Benign) for raw WebAssembly execution		[cite: 13, 14]
GenAI Chrome Extensions	341 Malicious GenAI extensions, 5,551 targeted corpus	Malicious (Public Disclosure vs. Internal Detection)	Publicly disclosed	Detailed in academic literature[15]

Acquisition Methodologies: Procuring Consecutive Extension Versions

Analyzing a single, isolated version of a malicious extension often provides an incomplete and potentially misleading picture of the threat landscape. Advanced malware developers actively utilize delayed activation patterns, timebombs, or conditional logic tied to specific target domains to remain dormant during the Chrome Web Store's automated vetting process[10]. These extensions only activate their payloads weeks after installation, heavily obfuscating their true nature upon initial upload[10]. Consequently, delta analysis—comparing the abstract syntax trees, API call sequences, and manifest permission changes between consecutive versions (version N and version $N+1$)—is the most reliable empirical method for detecting evasion and identifying the exact moment an extension is weaponized[9].

Procuring these consecutive versions, however, presents a significant technical hurdle. Major browser vendors intentionally do not expose historical version archives to the general public, as retaining easily accessible, outdated, and potentially vulnerable extensions poses severe security risks to users. Therefore, researchers must rely on specialized endpoints, documented APIs, and extensive third-party archives to systematically build datasets of consecutive versions.

The Chrome Update Endpoint (Omaha Protocol)

Google Chrome handles all extension updates via the Omaha protocol, communicating directly with the clients2.google.com update server. Researchers can manually interact with this endpoint to download raw, cryptographically signed .crx files directly from Google's servers[25]. To successfully acquire an extension, the HTTP GET request must be carefully formatted to spoof a legitimate Chrome client requesting an update. The standard URL structure relies on passing specific parameters: https://clients2.google.com/service/update2/crx?response=redirect&prodversion=[CHROME_VERSION]&acceptformat=crx2,crx3&x=id%3D[EXTENSION_ID]%26uc[1].

The mechanics of this request are critical. The prodversion parameter specifies the Chrome browser version (e.g., 114.0.5735.133 or a high placeholder like 9999), ensuring the server returns a compatible CRX file[1]. The acceptformat parameter is vital for modern compatibility;

Google transitioned to the CRX3 format to implement stronger cryptographic signatures and prevent header tampering, so passing crx2,crx3 ensures the server responds appropriately with the modern packaging format[25]. Finally, the x=id parameter passes the unique 32-character extension identifier[28]. A properly configured script must spoof the Chrome User-Agent and follow HTTP redirects to capture the binary payload[25].

Reliability for Delta Analysis: While this endpoint provides pristine, officially signed packages, its utility for historical delta analysis is severely limited. The Omaha protocol is designed to serve only the *latest* active version of an extension available in the store; it does not allow a user to pass a specific version parameter to pull historical files[1]. Therefore, to obtain consecutive versions utilizing this method, a researcher must build and maintain a continuous monitoring pipeline—polling the endpoint daily, downloading the CRX, and storing it chronologically.

Target Yield: This method yields highly reliable pairs of both **benign and malicious** extensions, but only if the active monitoring catches the malicious update in real-time before Google detects the threat and purges the extension from the Web Store.

Mozilla Add-ons (AMO) v5 API

In stark contrast to Google's opaque update mechanisms, Mozilla Firefox provides an incredibly transparent and developer-friendly REST API for interacting with the addons.mozilla.org (AMO) infrastructure[29]. The AMO v5 API allows researchers to programmatically query the entire version history of an extension with simple HTTP requests[31].

By sending a GET request to the specific endpoint:https://addons.mozilla.org/api/v5/addons/addon/{slug}/versions/ [cite: 32]

The server returns a comprehensive JSON payload detailing the complete version history of the specified add-on. This structured payload includes metadata on distribution channels, review dates, and, most importantly, direct download URLs for every signed XPI file associated with that extension[30]. A researcher can utilize a basic Python script to parse this JSON, extract the file.url property for each version, and sequentially download the XPI files, instantly yielding a perfect dataset of consecutive versions (e.g., v1.0, v1.1, v1.2) without the need for active daily polling[31].

Reliability for Delta Analysis: The AMO v5 API is the absolute gold standard for acquiring consecutive extension versions due to its transparency and ease of programmatic access.

Target Yield: This pipeline yields almost exclusively **benign** pairs. While it can occasionally yield malicious pairs if the add-on has not yet been fully purged from Mozilla's database, Mozilla actively revokes signatures and deletes historical records for known malware, making it difficult to scrape banned historical versions[31].

Third-Party Archives (crx4chrome and chrome-stats)

Because Google does not expose historical CRX downloads, third-party archiving services serve as indispensable repositories for retrospective delta research. Services such as crx4chrome and chrome-stats operate by actively mirroring Chrome Web Store updates, maintaining vast databases of historical CRX files even after they have been superseded or removed by Google[34].

Chrome-stats, in particular, is a robust analytical platform that provides deep metadata and historical tracking, utilizing a cross-store mapping process to track equivalent extensions across different ecosystems, such as Chrome and Microsoft Edge[3][4]. Academic researchers frequently rely on these third-party archives to assemble datasets of equivalent extensions and track their lifecycle changes longitudinally[5].

Reliability for Delta Analysis: These platforms are highly reliable for retrospective analysis.**Target Yield:** Third-party archives represent the most effective and reliable method for obtaining **historical malicious** updates. They allow researchers to download the exact malicious version of an extension alongside the benign version that preceded it, long after the extension has been banned and scrubbed from the official Chrome Web Store.

GitHub Releases for Open-Source Extensions

For foundational research focused on benign update patterns, code decay, software engineering practices, and the unintentional introduction of vulnerabilities, GitHub serves as an impeccable resource. Many highly popular browser extensions, such as uBlock Origin or Bitwarden, are fully open-source and maintain public repositories.

By utilizing the GitHub REST API to iterate through a repository's release tags, researchers can systematically download the exact source code for consecutive versions of an extension[6].

Reliability for Delta Analysis: While the downloaded artifacts are unpacked directories containing raw source code rather than officially compiled and cryptographically signed CRX or XPI files, they represent the purest form of data for localized delta analysis. **Target Yield:** This method yields exclusively **benign** pairs, or pairs demonstrating the introduction of unintentional vulnerabilities, as malicious actors do not publicly version control their malware on GitHub.

Academic Corpora: The 'You've Changed' Dataset

Rather than building a complex scraping infrastructure from scratch to monitor the Omaha protocol or third-party archives, researchers can directly leverage the pre-compiled *extensiondeltas* dataset from the 'You've Changed' CCS 2020 paper[8]. This corpus was generated precisely to solve the consecutive version acquisition problem for the broader research community. The authors maintained a dedicated crawling infrastructure that monitored and downloaded updates for hundreds of thousands of extensions over an extended period, ultimately cataloging 922,684 specific extension versions[8].

Reliability for Delta Analysis: This dataset is extremely reliable as the heavy lifting of continuous acquisition has already been completed. **Target Yield:** This is the most reliable academic source for acquiring **both benign and malicious** consecutive pairs. The researchers specifically designed the dataset to capture extensions *before* they turned malicious and *after* the update hijacked them, filtering for extensions that exhibited significant behavioral deviations between updates based on user rating anomalies[8].

Structured Synthesis of Version Acquisition Methodologies

Acquisition Method	Operational Mechanism /	Target Yield (Benign vs.	Reliability & Suitability for

	Protocol	Malicious)	Delta Analysis
Chrome Update Endpoint	Omaha protocol GET request to clients2.google.com mimicking a browser update[1]	Both (requires real-time capture)	**Low for retrospective analysis**. Only serves the *latest* active version; requires building infrastructure for continuous daily polling[1].
AMO v5 API (Firefox)	REST API GET request to /api/v5/addons/addon/{slug}/versions/ returning JSON[32]	Exclusively Benign	**Extremely High**. Natively provides direct download URLs for all historical XPI files for immediate sequential downloading[31].
crx4chrome / chrome-stats	Third-party web scraping, archiving databases, and cross-store metadata mapping[34]	Both (Strongest for Malicious)	**High**. The definitive source for retrieving older versions of malware that Google has already banned or forcibly updated[34].
GitHub Releases	API queries to open-source extension repositories	Exclusively Benign	**Perfect for source-code deltas**, but lacks the compiled CRX headers, Omaha packaging, and cryptographic signatures.
'You've Changed' Corpus	Academic dataset of 922,684 pre-scraped,	Both (Pre-classified)	**Extremely High**. A ready-made dataset specifically

	historical versions[9]		designed for longitudinal update delta analysis[8].

Third-Order Insights: Evolving Threat Paradigms and Analytical Evasion

The systematic compilation of these public datasets and the continuous acquisition of consecutive versions are merely the foundational prerequisites for browser extension security research. Analyzing this massive volume of code across temporal boundaries reveals profound shifts in how extension malware is constructed, how it actively evades detection, and how browser vendors are fundamentally altering the underlying ecosystem's architecture in a continuous arms race to counteract these threats.

The Manifest V3 (MV3) Security Shift and Malicious Adaptation

The transition from Manifest V2 (MV2) to Manifest V3 (MV3) represents the most significant architectural and philosophical disruption in the history of browser extensions[4]. Driven by imperatives to enhance user privacy, security, and browser performance, Google mandated MV3 to severely restrict the capabilities of highly privileged APIs that were historically abused by threat actors[7].

The most heavily debated and structurally impactful change is the deprecation of the webRequest API. Under MV2, this API allowed extensions to intercept, read, and programmatically modify network requests in real-time, essentially granting the extension Man-in-the-Middle capabilities[4]. MV3 replaces this with declarativeNetRequest, which forces developers to declare static blocking rules in advance, effectively blinding the extension to the actual data payloads and headers of the network requests[4]. Furthermore, MV3 outright bans the execution of remotely hosted code; all executable logic must be explicitly included within the extension's packaged CRX file, and background pages have been entirely replaced by ephemeral service workers that spin up and down based on events[7].

Initial empirical assessments of MV3 suggested a massive victory for browser security. A dynamic execution analysis of 517 confirmed malicious MV2 extensions demonstrated that MV3's restrictions successfully neutralized 87.8% of the APIs commonly abused by malware, rendering over 70% of legacy extension malware entirely non-functional post-conversion[4]. However, deeper analysis of recent datasets indicates that threat actors have already adapted to these new constraints.

Advanced malicious extensions in the MV3 era bypass the remote code execution bans by aggressively abusing the web_accessible_resources directive[4]. By declaring malicious scripts as web-accessible, extensions can inject them directly into the DOM of target pages, relying on the page's execution context to launch attacks like credential harvesting and DOM-based Cross-Site Scripting (XSS), retaining 56% of their original malicious capabilities[4]. Furthermore, by leveraging the chrome.runtime.connect API for external messaging, malicious service workers can establish covert, two-way communication channels with attacker-controlled web

pages, bypassing traditional static analysis engines that only look for explicit malicious code within the CRX file[1].

Defeating Evasion via Forced Execution Engines

As extensions are forced by MV3 to move their malicious logic into dynamically injected content scripts and conditionally executed service workers, static analysis of extension CRX datasets yields rapidly diminishing returns. Malware authors increasingly employ complex environmental checks, timebombs, and heavy JavaScript obfuscation (eval functions, string manipulations) to ensure their payloads remain dormant during the automated vetting processes of the Chrome Web Store[10].

To extract meaningful behavioral data from static corpora like *chrome-mal-ids* and the *You've Changed* dataset, researchers have pioneered the use of forced execution JavaScript engines. Tools such as **FV8** (a heavily modified version of the Chromium V8 JavaScript engine) are designed to forcefully traverse branch conditions in JavaScript code that would otherwise remain unexecuted under normal dynamic analysis[38]. FV8 intercepts APIs that conditionally inject dynamic code (such as setTimeout, setInterval, or asynchronous fetch requests) and forces their immediate execution, effectively neutralizing timebombs and bypassing logic designed to fingerprint the execution environment[24].

The empirical results of applying forced execution are staggering. When FV8 was applied to previously unexamined extensions, the engine achieved an 11% increase in code coverage over default V8, successfully executing 8.7 million lines of hidden code[24]. This deep traversal discovered over 16,000 injected third-party scripts and identified 110 confirmed malicious extensions across 28 distinct evasion categories (including Login evasion, Timebombs, and Fingerprinting) that had completely evaded all prior static and dynamic detection mechanisms[24].

Similarly, frameworks like **Fakeium** utilize the Node.js isolated-vm library in conjunction with dynamic Proxy mocks to simulate browser APIs[37]. By providing mocked responses to undefined properties requested by obfuscated malware, Fakeium tricks the extension into revealing its deobfuscated payload strings and intent without requiring the overhead of a full, resource-heavy instrumented browser environment[37].

The Pervasiveness of Code Duplication and Incremental Evasion

A critical third-order insight derived from the analysis of massive extension datasets is the overwhelming prevalence of code reuse and "incremental evasion" among threat actors. The Chrome Web Store's automated vetting process struggles significantly when an attacker uploads a slightly modified, obfuscated version of a previously banned malicious extension[11].

Research utilizing the **DeDup.js** framework demonstrated that the majority of files across the extension ecosystem are highly redundant[22]. By generating SHA-256 hashes and utilizing fuzzy similarity analysis across the unpacked static resources (HTML, CSS, JS) of extension datasets, DeDup.js uncovered a critical failure in current vetting paradigms: 86% of newly infringing extensions were structurally identical to items previously identified and banned by Google[11]. By cross-referencing new uploads against historical datasets of known malware, deduplication pipelines can identify and flag repackaged malware with near-zero false positives. This

methodology was instrumental in detecting the CacheFlow attack, identifying 53 extensions that were duplicating malicious behaviors while attempting to evade static vetting through minor structural alterations[22].

The Generative AI Vector: A Novel Execution Context

Finally, the datasets tracking GenAI extensions highlight a fundamental shift in extension utility and vulnerability[15]. Extensions powered by Large Language Models break the traditional boundaries of predictable, deterministic execution. Unlike static HTML manipulators, AI extensions process arbitrary, unstructured user content through remote API inferences, creating novel attack surfaces[17].

This integration introduces severe vulnerabilities, most notably prompt injection. If an AI summarizer extension reads a web page containing hidden, malicious prompt instructions—such as invisible text dictating the LLM to exfiltrate session tokens to an external server—the extension will blindly process the attack[17]. Empirical testing using the *Assure* framework on AI extensions found an average of 88.5 distinct security issues per extension, primarily due to the failure of developers to implement "visible-only processing" filters and robust input sanitization[17]. The integration of complex, non-deterministic AI features into the highly privileged browser environment guarantees that extension security research will require vastly more complex dynamic analysis datasets in the coming years, shifting the focus from static code analysis to behavioral monitoring of LLM interactions.

Concluding Analysis

The security of the browser ecosystem is inextricably linked to the rigorous, empirical analysis of extension behavior over time. The compendium of datasets presented in this report—ranging from the actively maintained, TPCI-verified IOCs in *chrome-mal-ids* to the massive historical deltas in the *You've Changed* corpus, and the bleeding-edge binary collections of *WasmMal-15K*—provides the critical foundational data required for this ongoing research.

Furthermore, the methodologies for acquiring consecutive extension versions highlight a stark and challenging contrast in vendor transparency. While Mozilla's AMO v5 API natively supports historical version retrieval through elegant REST endpoints, researching Chromium extensions necessitates complex workarounds involving the Omaha protocol, continuous polling infrastructures, and a heavy reliance on third-party archives like chrome-stats. As threat actors rapidly evolve their tactics to bypass Manifest V3 restrictions via web-accessible resources, exploit raw WebAssembly execution, and weaponize Generative AI interactions, the reliance on automated forced execution engines (like FV8) and deep delta analysis will only intensify. The future efficacy of browser extension security depends not just on algorithmic improvements in malware detection, but on the continued, open distribution and meticulous curation of these empirical datasets within the security research community.

Works cited

1. [HackerNotes Ep.95 & Ep.96] Cookies, Caching & Attacking Chrome, https://blog.criticalthinkingpodcast.io/p/cookies-caching-attacking-chrome-exte

nsions-with-matanber
2. UNPACKING DATA EXFILTRATION AND MALICIOUS BEHAVIOURS, https://arxiv.org/pdf/2512.10029
3. Towards Browser Controls to Protect Cookies from Malicious ... - arXiv, https://arxiv.org/html/2405.06830v1
4. Manifest V3 Unveiled: Navigating the New Era of Browser Extensions, https://arxiv.org/pdf/2404.08310
5. Developers Insight On Manifest v3 Privacy and Security ... - arXiv, https://arxiv.org/html/2507.13926v1
6. GitHub - The-Privacy-Commons-Institute/chrome-mal-ids: Effort to, https://github.com/mallorybowes/chrome-mal-ids
7. Manifest V3 Unveiled: Navigating the New Era of Browser Extensions, https://arxiv.org/html/2404.08310v1
8. Detecting Malicious Browser Extensions through their Update Deltas, http://www.mathcs.richmond.edu/~dszajda/classes/cs334/Spring_2021/papers/pantelaios_Detecting_Malicious_Browser_Extensions.pdf
9. wspr-ncsu/extensiondeltas - GitHub, https://github.com/wspr-ncsu/extensiondeltas
10. COMBAT: Continuous Monitoring of Browser Extensions against, https://www.atlantis-press.com/article/126024630.pdf
11. Detecting Malicious Browser Extensions through their Update Deltas, https://www.researchgate.net/publication/349829581_You've_Changed_Detecting_Malicious_Browser_Extensions_through_their_Update_Deltas
12. Comparative Study of Deep Learning Architectures for Malicious, https://publications.waset.org/10014514/comparative-study-of-deep-learning-architectures-for-malicious-browser-extension-detection-on-a-balanced-dataset
13. WasmGuard: Enhancing Web Security through Robust Raw-Binary, https://github.com/Yuxia-Sun/WasmGuard
14. WasmGuard: Enhancing Web Security through Robust Raw-Binary, https://openreview.net/pdf?id=xALBXUXnet
15. (PDF) Malicious GenAI Chrome Extensions: Unpacking Data, https://www.researchgate.net/publication/398602720_Malicious_GenAI_Chrome_Extensions_Unpacking_Data_Exfiltration_and_Malicious_Behaviours
16. Malicious GenAI Chrome Extensions: Unpacking Data Exfiltration, https://www.alphaxiv.org/abs/2512.10029
17. Assure: Metamorphic Testing for AI-powered Browser Extensions, https://arxiv.org/html/2507.05307v1
18. SmartShield is a powerful malware detection and secure ... - GitHub, https://github.com/akriti04gupta/SmartShield
19. Malicious-URL-Detection-Using-Machine-Learning - GitHub, https://github.com/md-hameem/Malicious-URL-Detection-Using-Machine-Learning
20. Best CRXCavator Alternative for Extension Security - Spin.AI, https://spin.ai/blog/best-crxcavator-alternative-browser-extension-risk-assessment/

21. M108Falcon/crx_checker: Analyze browser extensions for ... - GitHub, https://github.com/M108Falcon/crx_checker
22. DeDup.js: Discovering Malicious and Vulnerable Extensions by, https://www.scitepress.org/Papers/2022/109006/109006.pdf
23. nodesocket/node-dedup: Performs a poor man's file deduplication, https://github.com/nodesocket/node-dedup
24. A Forced Execution Engine for Malicious JavaScript Detection, https://www.researchgate.net/publication/324398494_JSForce_A_Forced_Execution_Engine_for_Malicious_JavaScript_Detection
25. akhileshthite/chrome-extension-fetch: A utility to download ... - GitHub, https://github.com/akhileshthite/chrome-extension-fetch
26. How to download a CRX file from the Chrome web store for a given, https://stackoverflow.com/questions/7184793/how-to-download-a-crx-file-from-the-chrome-web-store-for-a-given-id
27. CRX3 File Signature Verification Bypass via Embedded ZIP64, https://issues.chromium.org/41485950
28. ChromePluginCrx/README.md at main - GitHub, https://github.com/jas502n/ChromePluginCrx/blob/main/README.md
29. Signing API now available - Mozilla Add-ons Community Blog, https://blog.mozilla.org/addons/2015/11/20/signing-api-now-available/
30. How I Mastered Firefox Add-on Self-Distribution: The Complete, https://dredyson.com/how-i-mastered-firefox-add-on-self-distribution-the-complete-configuration-guide-for-pros-fixing-broken-links-hidden-url-workarounds-and-what-every-developer-needs-to-know-about-amos-di/
31. How to Programmatically Install Firefox Extensions (And Why It, https://dev.to/alanwest/how-to-programmatically-install-firefox-extensions-and-why-it-breaks-2b01
32. A new API for submitting and updating add-ons - The Mozilla Blog, https://blog.mozilla.org/addons/2022/03/17/new-api-for-submitting-and-updating-add-ons/
33. Mozilla-Firefox/Add_Firefox_Extensions.ps1 at master - GitHub, https://github.com/farag2/Mozilla-Firefox/blob/master/Add_Firefox_Extensions.ps1
34. Analysing Browser Extensions Across Stores - Aurore Fass, https://aurore54f.github.io/papers/2026-madweb_chaudhry.pdf
35. Developers Insight On Manifest v3 Privacy and Security ... - arXiv, https://arxiv.org/pdf/2507.13926
36. Developers' Insight on Manifest v3 Privacy and Security, https://www.scitepress.org/Papers/2025/136690/136690.pdf
37. Fakeium: A Dynamic Execution Environment for JavaScript Program, https://arxiv.org/html/2410.20862v1
38. FV8: A Forced Execution JavaScript Engine for Detecting Evasive, https://www.usenix.org/system/files/usenixsecurity24-pantelaios.pdf
39. FV8: A Forced Execution JavaScript Engine for Detecting Evasive, https://arxiv.org/abs/2405.13175
40. [Literature Review] Fakeium: A Dynamic Execution Environment for,

https://www.themoonlight.io/en/review/fakeium-a-dynamic-execution-environment-for-javascript-program-analysis